

CSA0604-DESIGN AND ANALYSIS OF ALGORITHM

NAME : SANTHIYA S

REG. NO : 192511360

Session 2 Lab Programs

1. Write a program that takes an input list of n numbers and creates a new list containing only the unique elements from the original list. What is the space complexity of the algorithm?

CODE:

```
#include <stdio.h>

int main() {

    int n, i, j, k = 0;

    int a[100], unique[100];

    int found;

    printf("Enter the number of elements: ");

    scanf("%d", &n);

    printf("Enter the elements:\n");

    for (i = 0; i < n; i++) {

        scanf("%d", &a[i]);

    }

    // Find unique elements

    for (i = 0; i < n; i++) {

        found = 0;

        for (j = 0; j < k; j++) {

            if (a[i] == unique[j]) {

                found = 1;
```

```

        break;

    }

}

if (found == 0) {

    unique[k] = a[i];

    k++;

}

}

printf("Unique elements: ");

for (i = 0; i < k; i++) {

    printf("%d ", unique[i]);

}

return 0;

}

```

OUTPUT:

main.c	Run ▶	Output
<pre> 1 #include <stdio.h> 2 int main() { 3 int n, i, j, k = 0; 4 int a[100], unique[100]; 5 int found; 6 printf("Enter the number of elements: "); 7 scanf("%d", &n); 8 printf("Enter the elements:\n"); 9 for (i = 0; i < n; i++) { 10 scanf("%d", &a[i]); 11 } 12 for (i = 0; i < n; i++) { 13 found = 0; 14 for (j = 0; j < k; j++) { 15 if (a[i] == unique[j]) { 16 found = 1; 17 break; 18 } 19 } 20 if (found == 0) { 21 unique[k] = a[i]; 22 k++; </pre>		<pre> Enter the number of elements: 7 Enter the elements: 2 3 4 5 3 4 7 8 Unique elements: 2 3 4 5 7 </pre>

2.Sort an array of integers using the bubble sort technique. Analyze its time complexity using Big-O notation. Write the code

CODE:

```
#include <stdio.h>

int main() {

    int a[100], n, i, j, temp;

    printf("Enter the number of elements: ");

    scanf("%d", &n);

    printf("Enter the elements:\n");

    for (i = 0; i < n; i++) {

        scanf("%d", &a[i]);

    }

    // Bubble Sort

    for (i = 0; i < n - 1; i++) {

        for (j = 0; j < n - i - 1; j++) {

            if (a[j] > a[j + 1]) {

                temp = a[j];

                a[j] = a[j + 1];

                a[j + 1] = temp;

            }

        }

    }

    printf("Sorted array: ");

    for (i = 0; i < n; i++) {
```

```

        printf("%d ", a[i]);

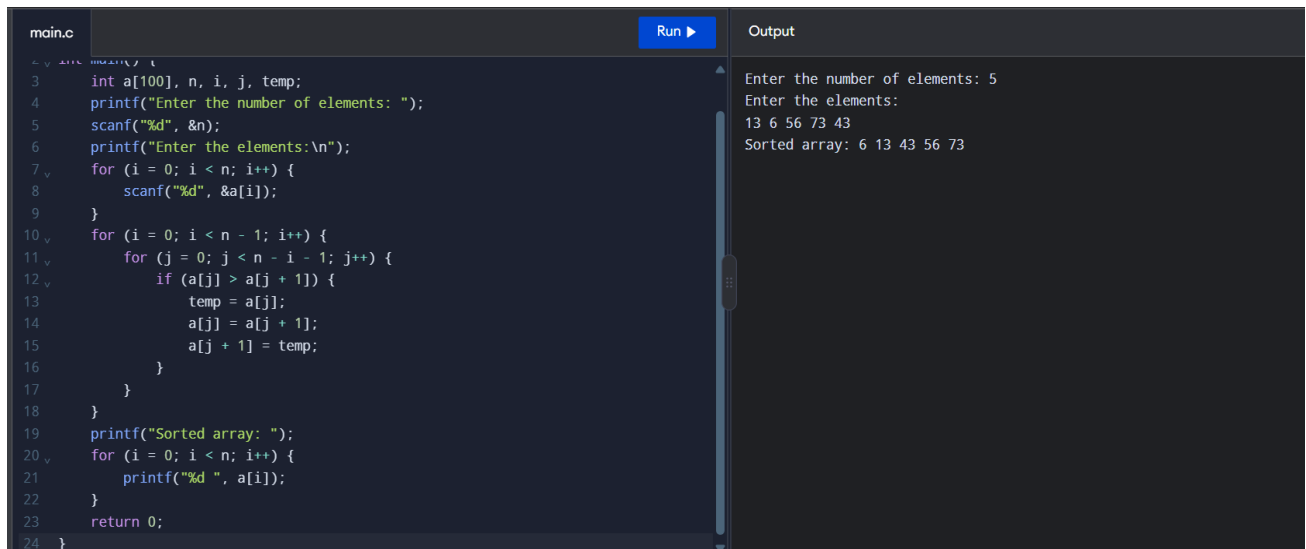
    }

    return 0;

}

```

OUTPUT:



```

main.c
3   int a[100], n, i, j, temp;
4   printf("Enter the number of elements: ");
5   scanf("%d", &n);
6   printf("Enter the elements:\n");
7   for (i = 0; i < n; i++) {
8       scanf("%d", &a[i]);
9   }
10  for (i = 0; i < n - 1; i++) {
11      for (j = 0; j < n - i - 1; j++) {
12          if (a[j] > a[j + 1]) {
13              temp = a[j];
14              a[j] = a[j + 1];
15              a[j + 1] = temp;
16          }
17      }
18  }
19  printf("Sorted array: ");
20  for (i = 0; i < n; i++) {
21      printf("%d ", a[i]);
22  }
23  return 0;
24 }

```

Run

Output

```

Enter the number of elements: 5
Enter the elements:
13 6 56 73 43
Sorted array: 6 13 43 56 73

```

3. Checks if a given number x exists in a sorted array arr using binary search. Analyze its time complexity using Big-O notation.

CODE:

```

#include <stdio.h>

int main() {

    int arr[100], n, x;

    int low, high, mid;

    int found = 0;

    printf("Enter the number of elements: ");

    scanf("%d", &n);

    printf("Enter the sorted elements:\n");

```

```
for (int i = 0; i < n; i++) {  
    scanf("%d", &arr[i]);  
}  
  
printf("Enter the number to search: ");  
  
scanf("%d", &x);  
  
low = 0;  
  
high = n - 1;  
  
while (low <= high) {  
    mid = (low + high) / 2;  
  
    if (arr[mid] == x) {  
        found = 1;  
        break;  
    }  
  
    else if (arr[mid] < x) {  
        low = mid + 1;  
    }  
  
    else {  
        high = mid - 1;  
    }  
}  
  
if (found)  
    printf("%d is found in the array.\n", x);  
  
else  
    printf("%d is not found in the array.\n", x);
```

```

    return 0;

}

```

OUTPUT:

```

main.c Run ▶ Output
1 #include <stdio.h>
2 int main() {
3     int arr[100], n, x;
4     int low, high, mid;
5     int found = 0;
6     printf("Enter the number of elements: ");
7     scanf("%d", &n);
8     printf("Enter the sorted elements:\n");
9     for (int i = 0; i < n; i++) {
10         scanf("%d", &arr[i]);
11     }
12     printf("Enter the number to search: ");
13     scanf("%d", &x);
14     low = 0;
15     high = n - 1;
16     while (low <= high) {
17         mid = (low + high) / 2;
18         if (arr[mid] == x) {
19             found = 1;
20             break;
21         }
22         else if (arr[mid] < x) {

```

Enter the number of elements: 7
Enter the sorted elements:
2 5 7 18 53 66 43
Enter the number to search: 66
66 is found in the array.

4. Given an array of integers nums, sort the array in ascending order and return it. You must solve the problem without using any built-in functions in $O(n\log(n))$ time complexity and with the smallest space complexity possible.

CODE:

```

#include <stdio.h>

// Heapify the array

void heapify(int arr[], int n, int i) {

    int largest = i;

    int left = 2 * i + 1;

    int right = 2 * i + 2;

    int temp;

    if (left < n && arr[left] > arr[largest])

        largest = left;

```

```

    if (right < n && arr[right] > arr[largest])

        largest = right;

    if (largest != i) {

        temp = arr[i];

        arr[i] = arr[largest];

        arr[largest] = temp;

        heapify(arr, n, largest);

    }

}

```

// Heap Sort

```

void heapSort(int arr[], int n) {

    int i, temp;

    // Build max heap

    for (i = n / 2 - 1; i >= 0; i--)

        heapify(arr, n, i);

    // Extract elements one by one

    for (i = n - 1; i > 0; i--) {

        temp = arr[0];

        arr[0] = arr[i];

        arr[i] = temp;

        heapify(arr, i, 0);

    }

}

int main() {

```

```

int nums[100], n, i;

printf("Enter the number of elements: ");

scanf("%d", &n);

printf("Enter the elements:\n");

for (i = 0; i < n; i++) {

    scanf("%d", &nums[i]);

}

heapSort(nums, n);

printf("Sorted array: ");

for (i = 0; i < n; i++) {

    printf("%d ", nums[i]);

}

return 0;

}

```

OUTPUT:

main.c	Run ▶	Output
<pre> 1 #include <stdio.h> 2 void heapify(int arr[], int n, int i) { 3 int largest = i; 4 int left = 2 * i + 1; 5 int right = 2 * i + 2; 6 int temp; 7 if (left < n && arr[left] > arr[largest]) 8 largest = left; 9 if (right < n && arr[right] > arr[largest]) 10 largest = right; 11 if (largest != i) { 12 temp = arr[i]; 13 arr[i] = arr[largest]; 14 arr[largest] = temp; 15 heapify(arr, n, largest); 16 } 17 } 18 void heapSort(int arr[], int n) { 19 int i, temp; 20 for (i = n / 2 - 1; i >= 0; i--) 21 heapify(arr, n, i); 22 for (i = n - 1; i > 0; i--) { </pre>		<pre> Enter the number of elements: 5 Enter the elements: 3 46 88 31 72 Sorted array: 3 31 46 72 88 </pre>